

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB RECORD

Computer Network Lab (23CS5PCCON)

Submitted by

Hitha Harish (1BM23CS115)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Academic Year 2025-26 (even)

B.M.S. College of Engineering

Bull Temple Road, Bangalore 560019

(Affiliated To Visvesvaraya Technological University, Belgaum)

Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “ Computer Network (23CS5PCCON)” carried out by **Hitha Harish (1BM23CS115)**, who is bonafide student of **B.M.S. College of Engineering**. It is in partial fulfilment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum. The Lab report has been approved as it satisfies the academic requirements of the above-mentioned subject and the work prescribed for the said degree.

Rashmi H Assistant Professor Department of CSE, BMSCE	Dr. Selva Kumar Professor & HOD Department of CSE, BMSCE
---	--

Index

Sl. No.	Date	Experiment Title	Page No.
1	13-08-2025	Simple PDU from source to destination using hub and switch as connecting devices.	4-8
2	17-09-2025	Default route and static route to the Router	9-13
3	03-09-2025	DHCP within a LAN and outside LAN	14-15
4	10-09-2025	Web Server, DNS within a LAN	16-18
5	08-10-2025	Operation of TELNET to access the router in server room from a PC in IT office	19-21
6	19-11-2025	RIP routing Protocol in Routers	22-23
7	15-10-2025	VLAN to make the PCs communicate among a VLAN	24-25
8	15-10-2025	WLAN to make the nodes communicate wirelessly	26-27
9	19-11-2025	Simple LAN to understand the concept and operation of ARP	28-29
10	08-10-2025	OSPF routing protocol	30-31
11	15-10-2025	TTL/ Life of a Packet	32-33
12	15-10-2026	Ping responses, destination unreachable, request timed out, reply	34-36
13	29-10-2025	Congestion control using Leaky bucket algorithm	37-38
14	29-10-2025	Error detecting code using CRC-CCITT	39-40
15	12-11-2025	TCP File Request–Response Using Client–Server Socket Program	41-42
16	12-11-2025	UDP File Request–Response Using Client–Server Socket Program	43-44

GithubLink: <https://github.com/HithaHarishCS/CN>

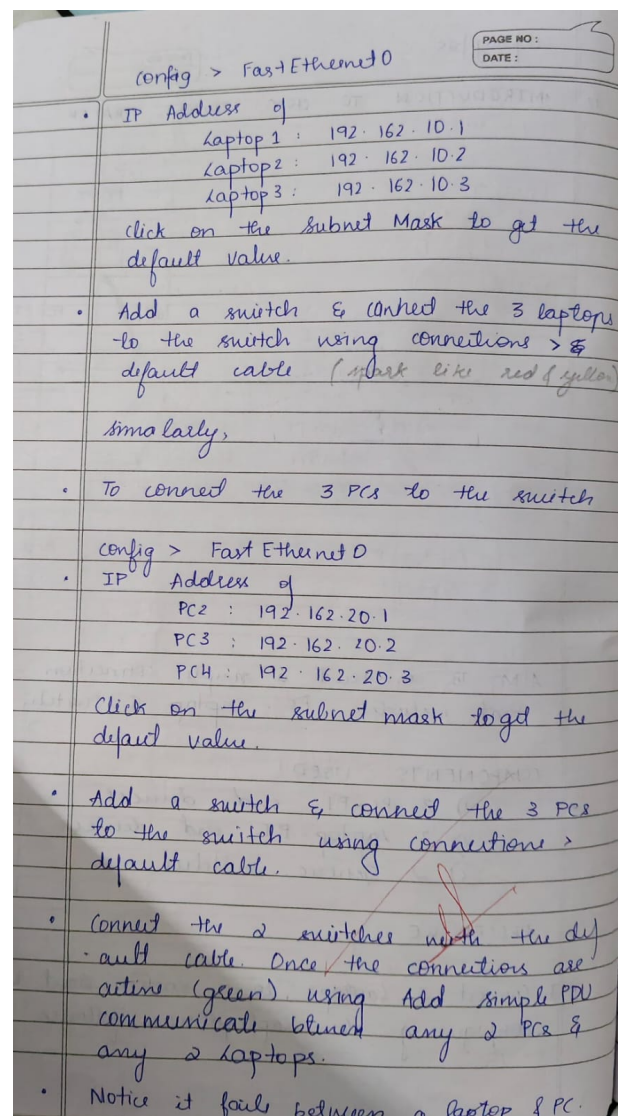
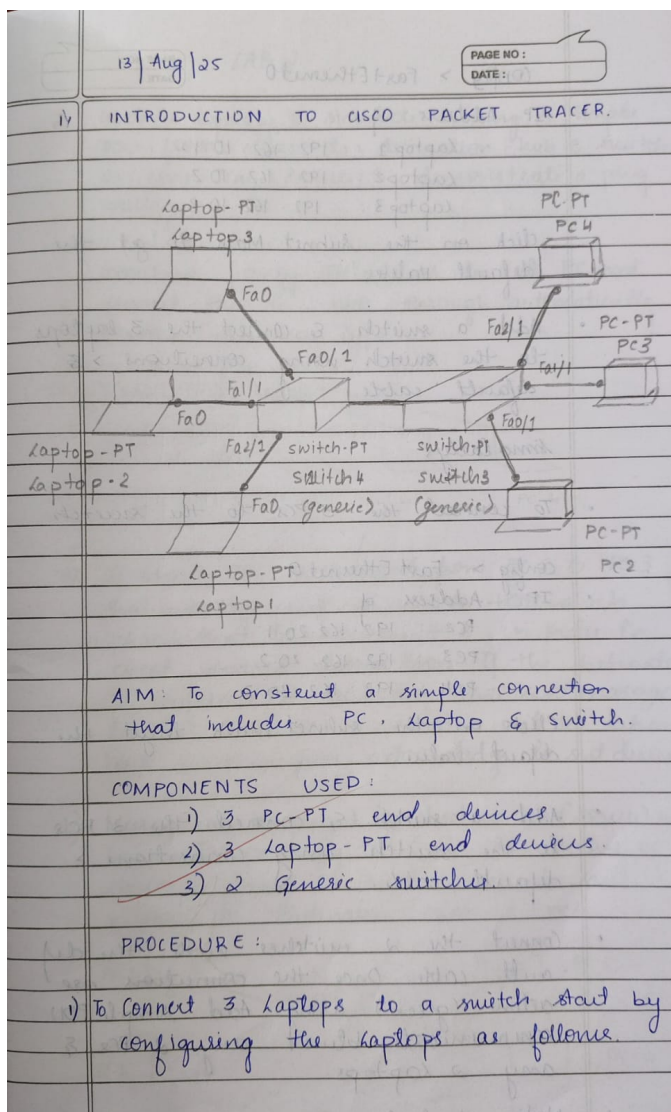
CYCLE 1:

Program 1

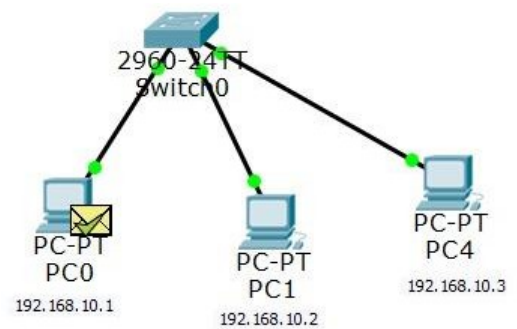
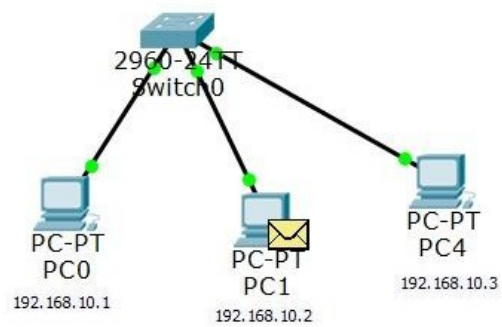
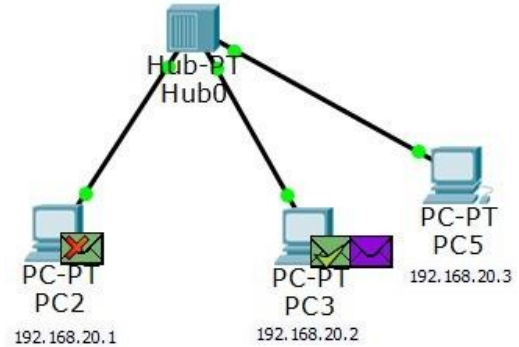
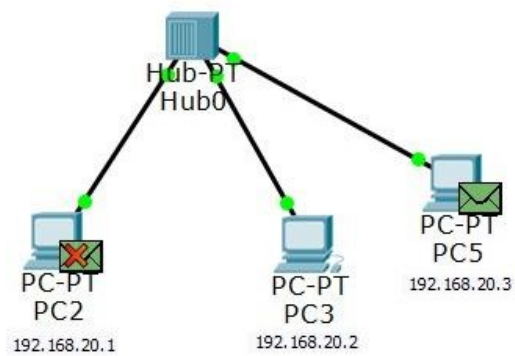
Aim of the program:

Create a topology and simulate sending a simple PDU from source to destination using hub and switch as connecting devices and demonstrate ping message.

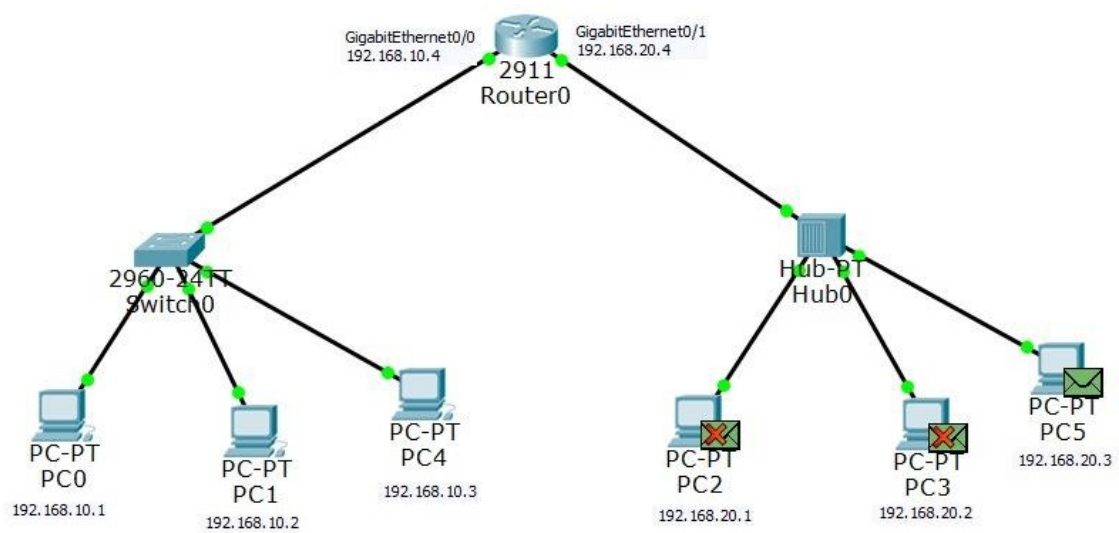
Procedure and topology:

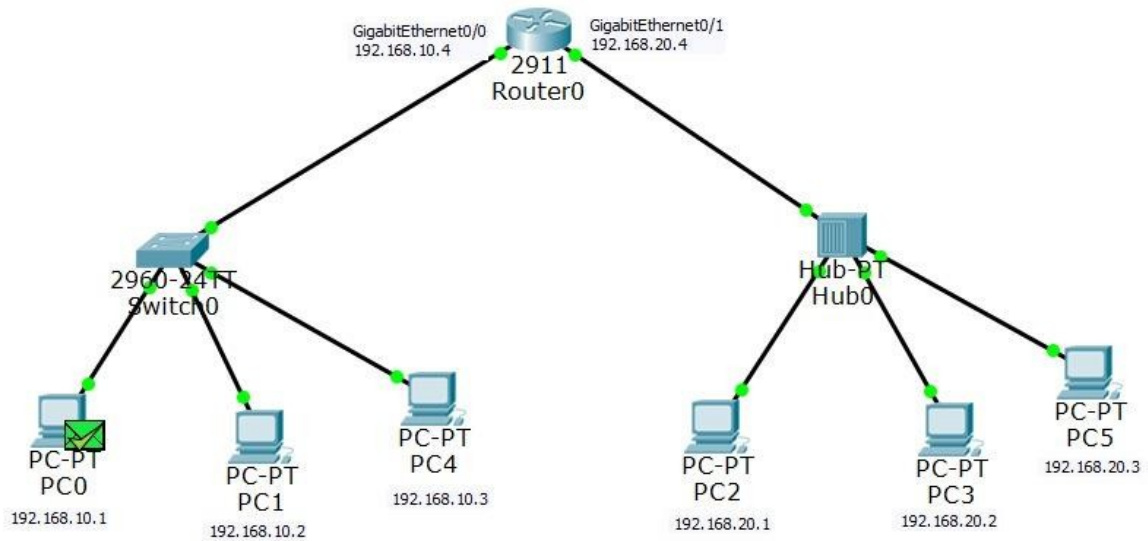


Screenshots/ Output:



Updated topology





Observation:

- In the hub-based topology, the PDU was broadcast to all ports, while the switch forwarded the PDU only to the destination MAC after learning addresses from incoming frames.
- Successful ICMP echo and echo-reply messages confirm that both devices enabled connectivity, with the switch demonstrating selective unicast forwarding and reduced unnecessary traffic.

Program 2

Aim of the program:

Configure default route, static route to the Router

Procedure and topology:

PAGE NO:
 DATE:

Observations: All the config PCs are connected to switches where these ports are filled with connected PCs when message sent from PC-1 to PC-3 it will be first sent to switch and that message will be reached to only destined PC unlike HUB such that message won't reach any of undesired PCs.

PC-1 PC-2 (192.168.12.2)
S1 S2
PC-3 PC-4
S3 S4

When PDU sent from PC-3 to PC-4 first it will be sent to S3 & then it reaches S4 & the message will be sent to desired destination.

12/10/20

PAGE NO:
 DATE:

LAB 2

2. Create a topology & simulate sending a simple PDU from source to destination hub & switch as connecting devices & demonstrate a ping message.

Connections: config IP address to all PC and connect this to HUB through automatically chosen connection type.

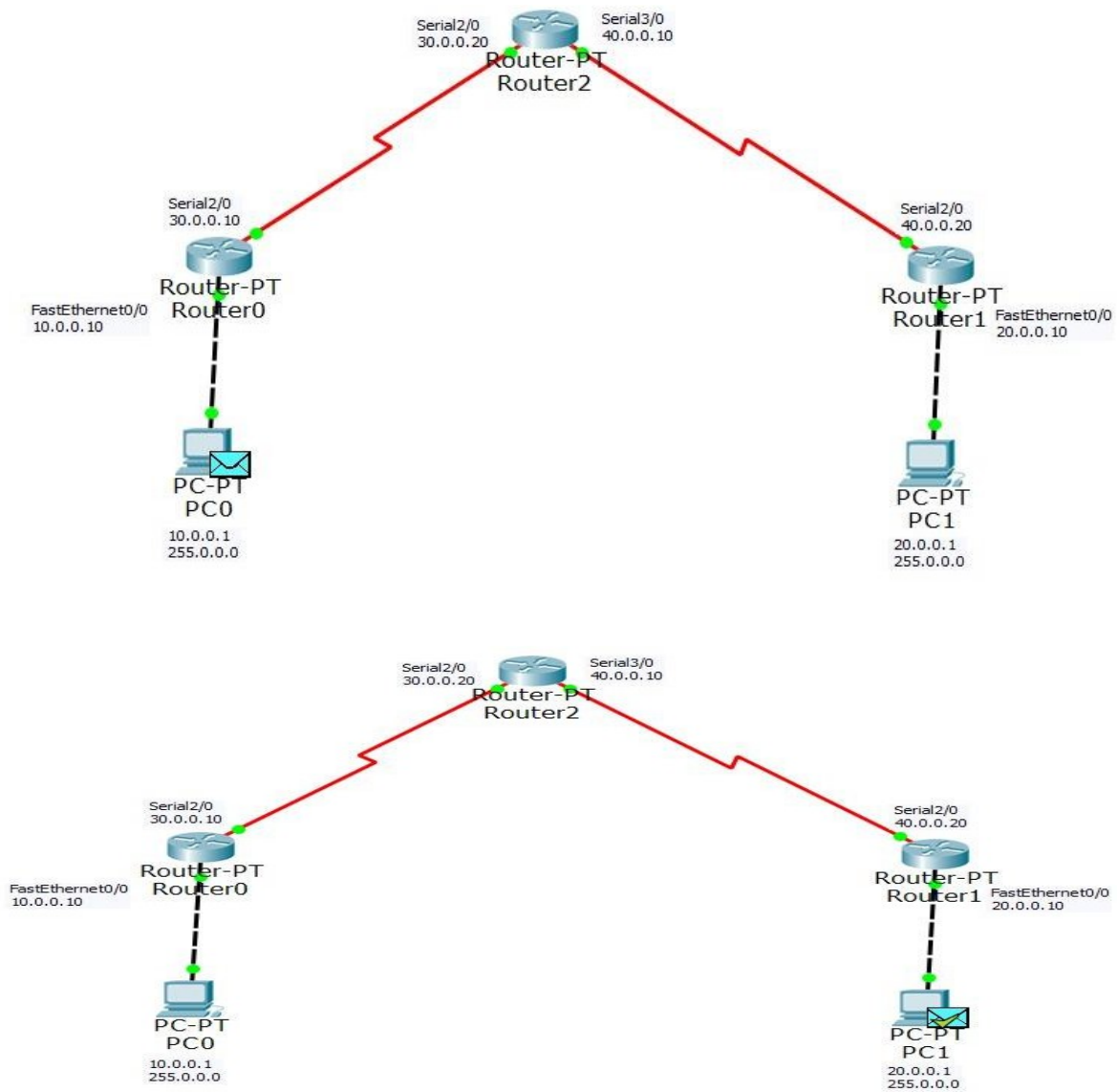
(192.168.10.1) PC-1 PC-2 PC-3 PC-4
HUB

a) If simple PDU is dropped from PC-1 to PC-3 that message will be sent to HUB which passes that to all the PCs in order to check desired destination. If the destination is confirmed it will drop that sent message to it, while other messages will be taken back & then give acknowledge to send device.

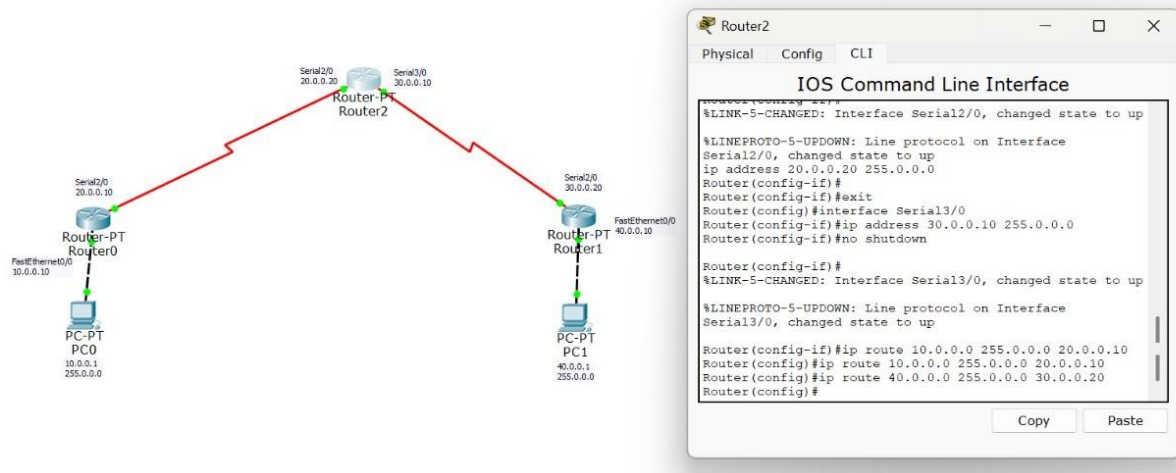
b) Message can be pinged by checking PC1 (source) and in cmd, ip config to check the status of source & ping 192.168.10.4 to send message to destination that is PC4.

PC-1 PC-3
PC-2 PC-4
switch

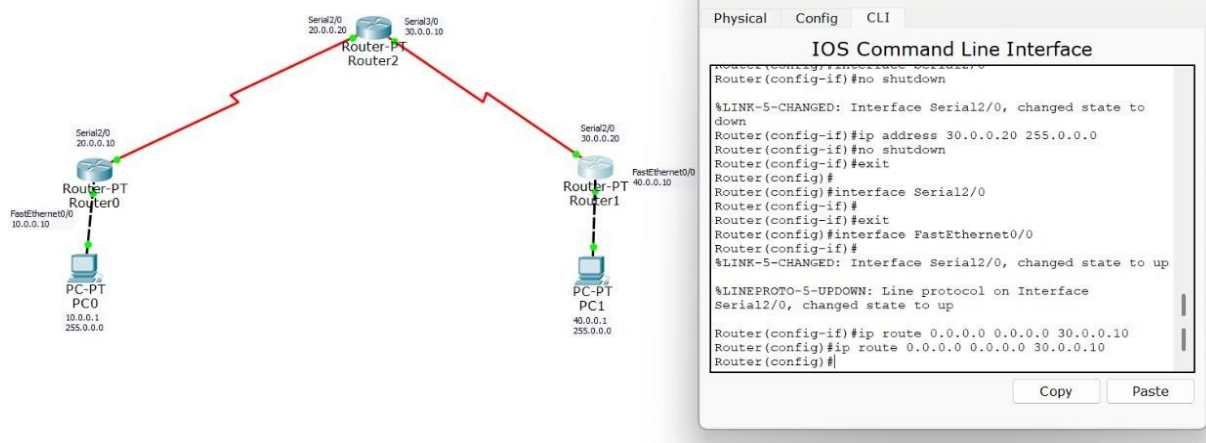
Screenshots/ Output:



Static routing CLI commands:



Default routing CLI commands:



Observation:

- The configured static and default routes correctly updated the router's routing table, enabling deterministic next-hop selection for remote networks.
- Successful ping tests verified that traffic was forwarded according to the static/default route entries, ensuring end-to-end reachability across different network segments.

Program 3

Aim of the program:

Configure DHCP within a LAN and outside LAN.

Procedure and topology:

PAGE NO:
 DATE:

3. Config DHCP within a LAN & outside LAN.

Create the following topology.

Router

Switch1

Switch2

server

laptop 2

PC0

PC2

PC3

On Server

- Navigate to DHCP
- Navigate to Desktop
- Config IP → IP configuration
- Under static
- Enter

IP Address : 192.168.10.2
Subnet Mask : 255.255.255.0
Default Gateway : 192.168.10.1

- Navigate from Desktop to Services
- Under DHCP

SWITCH 1 (On)

PoolName : switch1
Default Gateway : 192.168.10.1
DNS Server : 0.0.0.0

PAGE NO:
 DATE:

Start IP Address : 192.168.10.3
Subnet Mask : 255.255.255.0
Maximum number of users = 20
Add & save.

SWITCH2 (On)

PoolName : switch2.
Default Gateway : 192.168.20.1
DNS server : 0.0.0.0
Start IP Address : 192.168.20.2
Subnet Mask : 255.255.255.0
Maximum number of users : 20
Add & save.

- Click on Router
- Navigate to CLI
- continue with configuration debug : no
- ENTER
- Router> enable
- Router# conf t
- int Fa0/0
- ip address 192.168.10.1 255.255.255.0
- ip helper-address 192.168.10.2
- no shutdown
- do write memory
- exit

Similarly.

- int Fa4/0
- ip address 192.168.20.1 255.255.255.0

PAGE NO:
 DATE:

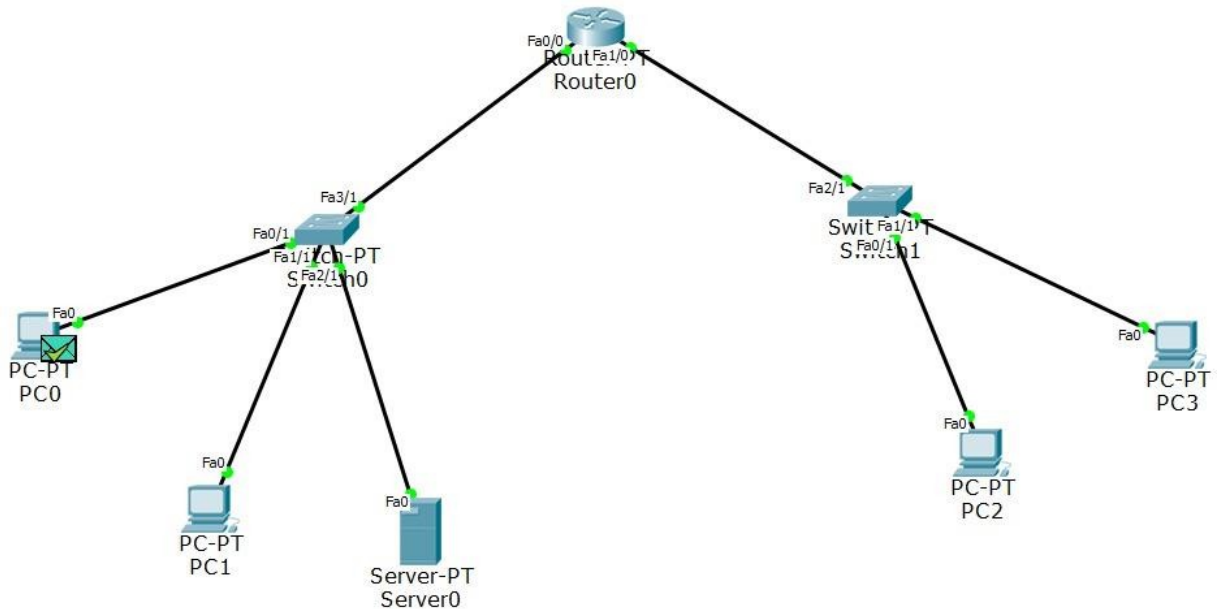
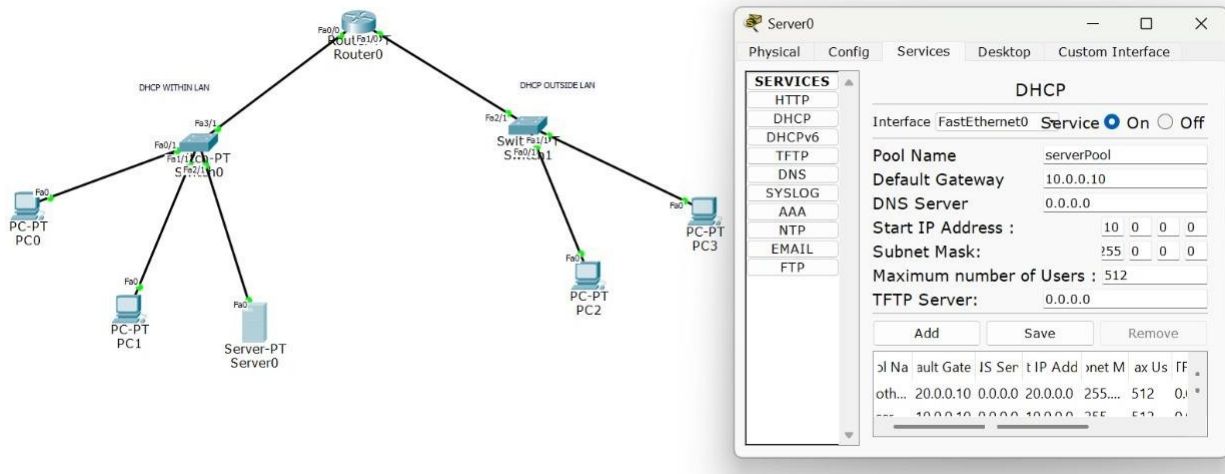
ip helper-address 192.168.10.2
no shutdown
do write memory
exit

VERIFICATION

PC / laptop under switch1
Desktop
DHCP
The OP : DHCP request successful.

3/4/25

Screenshots/ Output:



Observation:

- The DHCP server successfully allocated IP addresses to clients within the LAN, confirming proper scope configuration and automatic distribution of network parameters.
- DHCP relay (IP Helper) enabled clients outside the LAN to obtain leases from the central DHCP server, demonstrating correct inter-network forwarding of DHCP Discover and Offer messages.

Program 4

Aim of the program:

Configure Web Server, DNS within a LAN.

Procedure and topology:

PAGE NO:
 DATE:

Web-server0
IP Address: 192.168.1.6
Subnet Mask: 255.255.255.0
DNS Server: 192.168.1.5

DNS-Server
IP Address: 192.168.1.5
Subnet Mask: 255.255.255.0
DNS Server: 192.168.1.5

click on web server turn on HTTP & HTTPS
under File manager edit helloworld.html
change that content & save.

click on DNS server select DNS under service
select DNS under that turn of DNS service
Name: www.ltslearn.com
Address: 192.168.1.6

Finally check the connectivity
select PC-0 → Desktop → cmd.
Ping 192.168.1.5 [IP Address of DNS]
[Reply will be visible]
Ping 192.168.1.6 [IP address of webserver]
[Reply will be visible]

Under Desktop → web browser.
url: www.ltslearn.com
select small page → Actual the message
will be displayed on the screen.
This status that PCs are connected
successfully via switch to the server.

10/9/25.
 PAGE NO:
 DATE:

4. Configure webserver, DNS within a LAN.
b) Configure ip-address to router in packet
trace. Explain the following messages:
→ Ping response.
→ Destination unreachable.
→ Request Timeout.
→ Reply.

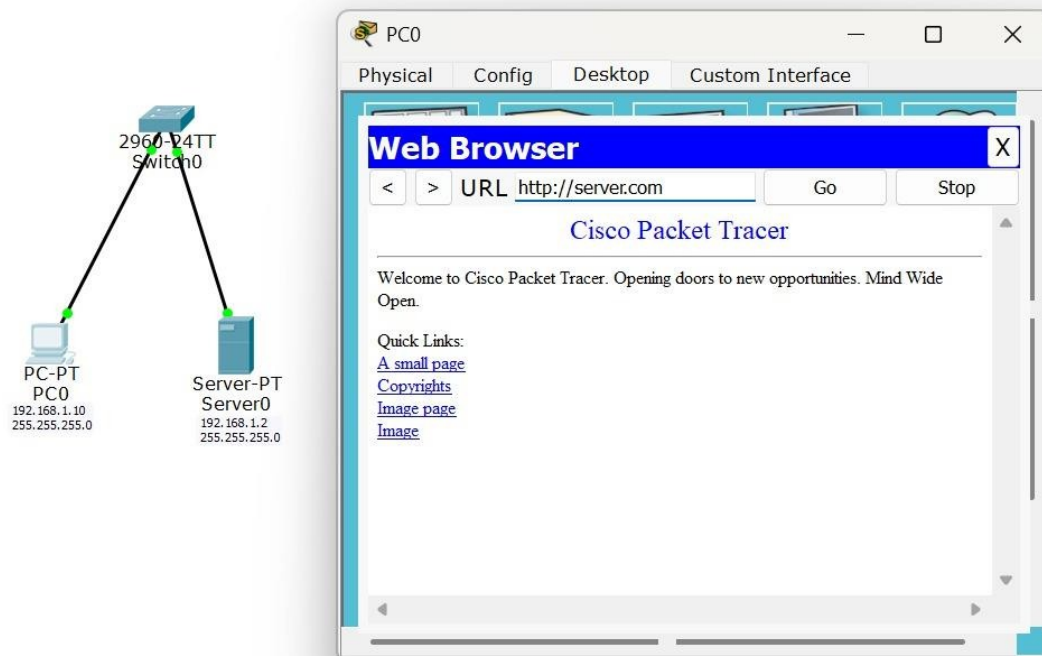
DNS is used to connect url to ip address.

```
graph TD
    Router((Router)) --- WebServer[Web Server server-PT]
    Router --- DNSServer[DNS Server server-PT]
    WebServer --- Switch[Switch]
    DNSServer --- Switch
    Switch --- PC0[PC-PT PC0]
    Switch --- PC1[PC-PT PC1]
```

• Config PC0 by selecting Desktop:
IP Address: 192.168.1.100
Subnet Mask: 255.255.255.0
DNS Server: 192.168.1.5

• Config PC1 by selecting Desktop:
IP Address: 192.168.1.101
Subnet Mask: 255.255.255.0
DNS Server: 192.168.1.5

Screenshots/ Output:



Observation:

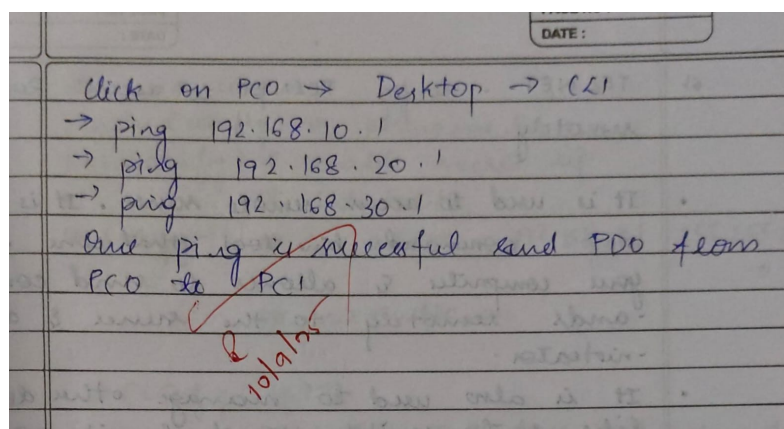
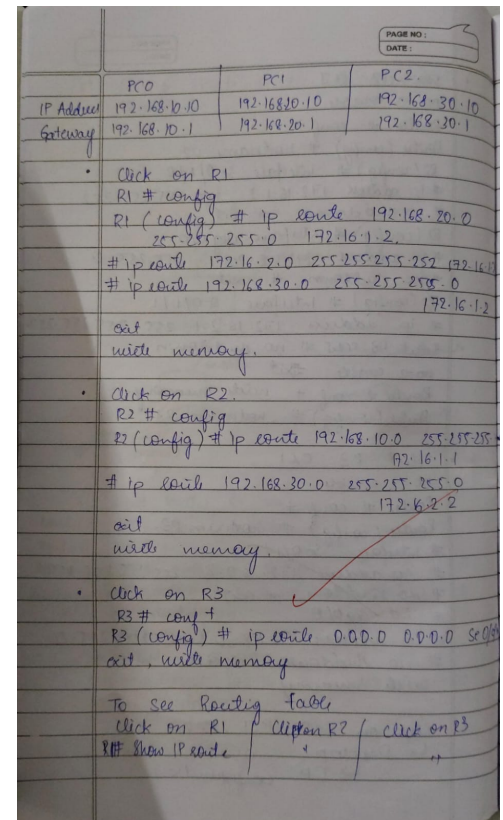
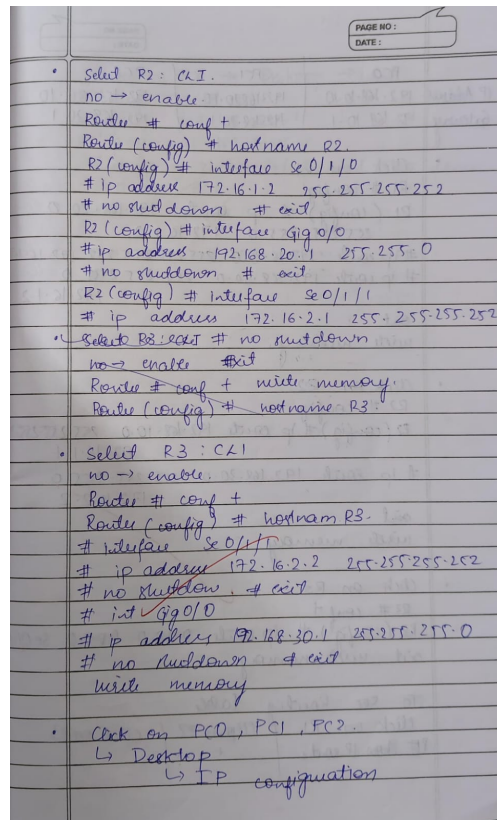
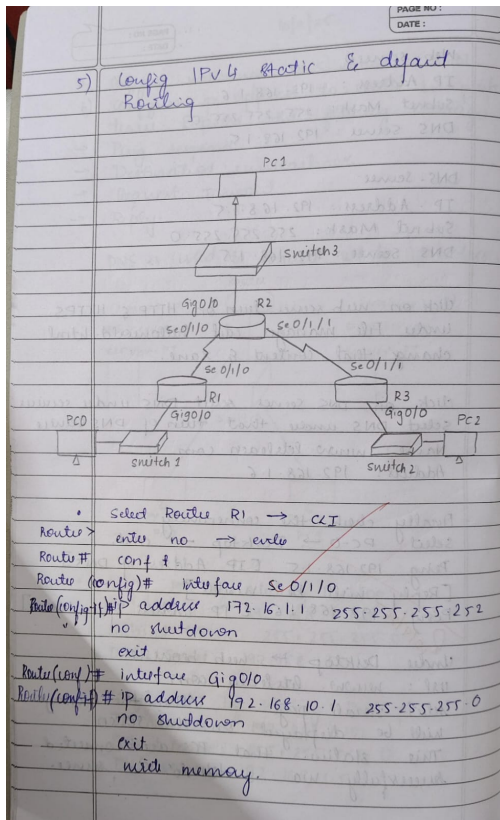
- The DNS server successfully resolved domain names to the corresponding web server's IP address, confirming proper hostname-to-IP mapping within the LAN.
- HTTP requests reached the web server using the DNS-resolved address, validating correct server configuration and internal LAN communication.

Program 5

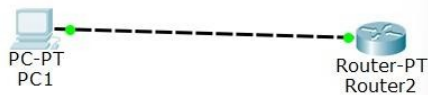
Aim of the program:

To understand the operation of TELNET by accessing the router in server room from a PC in IT office

Procedure and topology:



Screenshots/ Output:



A screenshot of the PC1 Command Prompt window. The window has tabs for 'Physical', 'Config', 'Desktop', and 'Custom Interface'. The 'Config' tab is active. The command prompt shows the following text:

```
Packet Tracer PC Command Line 1.0
PC>telnet 192.168.1.2
Trying 192.168.1.2 ...Open

User Access Verification

Password:
Router1>ping 192.168.1.1

Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 192.168.1.1, timeout
is 2 seconds:
!!!!
Success rate is 100 percent (5/5), round-trip
min/avg/max = 0/1/3 ms

Router1>exit

[Connection to 192.168.1.2 closed by foreign host]
PC>
```

A screenshot of the Router2 CLI window. The window has tabs for 'Physical', 'Config', and 'CLI'. The 'CLI' tab is active. The text shows the following commands and output:

```
Router2
Physical Config CLI

IOS Command Line Interface

Router(config-if)#ip address 192.168.1.2 255.255.255.0
Router(config-if)#npno shutdown

Router(config-if)#
%LINK-5-CHANGED: Interface FastEthernet0/0, changed state
to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface
FastEthernet0/0, changed state to up

Router(config-if)#exit
Router(config)#hostname Router1
Router1(config)#enable secret p1
Router1(config)#line vty 0 4
Router1(config-line)#login
% Login disabled on line 132, until 'password' is set
% Login disabled on line 133, until 'password' is set
% Login disabled on line 134, until 'password' is set
% Login disabled on line 135, until 'password' is set
% Login disabled on line 136, until 'password' is set
Router1(config-line)#password cisco
Router1(config-line)#exit
```

At the bottom of the window, there are 'Copy' and 'Paste' buttons.

Observation:

- The Telnet session successfully established a remote CLI connection to the router, confirming proper VTY line configuration and IP reachability between the IT office PC and the server room router.
- Command execution over the Telnet session demonstrated reliable remote device management.

Program 6

Aim of the program:

Configure RIP routing Protocol in Routers

Procedure and topology:

8/10/25

TEENET - Config TEENET to access Router remotely.

- It is used to access remote server. It is a simple command line tool that runs on your computer & allows to send commands remotely to the server & administration.
- It is also used to manage other devices like router, switch to check if process is open/closed on the server.

PC0

Desktop > IP configuration

IP Address: 192.168.1.2

Subnet Mask: 255.255.255.0

Default Gateway: 192.168.1.1

Router

> no

Enter

> enable

Router # conf t

Router (config) # hostname R1

R1 (config) # enable secret up

int Fa0/0

R1 (config-if) # ip address 192.168.1.1 255.255.255.0

no shutdown

R1 (config-if) #

%LINK-5-CHANGED: Interface FastEthernet0/0, changed state to up

%LINEPROTO-5-UPDOWN:

R1 (config-if) # line vty 0 5

R1 (config-line) # login

login disabled on line 194, until password is set

... 195 ... 196 ... 197 ... 198 ... 199

R1 (config-line) # password tp

R1 (config-line) # exit

R1 (config) # exit

R1 #

%SYS-5-CONFIG-I: Configured from console by console

wa

Building configuration ...

[OK]

R1 # show ip interface brief

Interface IP Address OK? Method Status

① Interface

Interface	IP Address	Manual	Up
FastEthernet0/0	192.168.1.1	YES	up
FastEthernet0/1	unassigned	YES	down
Vlan1	unassigned	YES	down

PC0 > desktop > Command Prompt

PC > ping 192.168.1.1

Pinging 192.168.1.1 ...

Reply from 192.168.1.1: bytes=32 time=0m

Ping statistics:

PC > telnet 192.168.1.1

Telnet 192.168.1.1 ... Open

User Access Verification

Password:

R1 > enable

Password:

R1 # show ip interface brief

(same OP as ①)

R1 # enable

R1 # conf t

R1 (config) # int Fa0/1

R1 (config-if) # ip address 192.168.1.2 255.255.255.0

% 192.168.1.0 overlaps with FastEthernet0/0

R1 (config-if) #

[connection to 192.168.1.1 closed by foreign host]

PC >

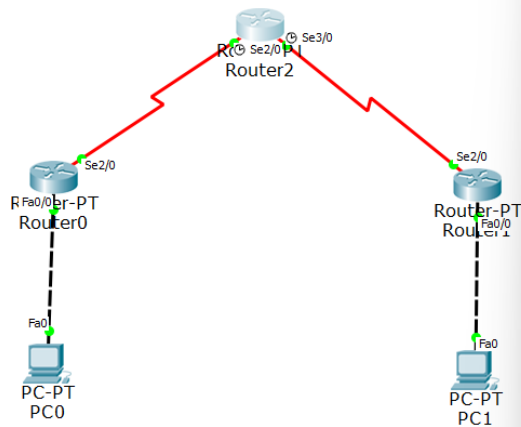
Router > CLI

R1 # show ip interface brief

② Interface IP Address OK? Method Status

Interface	IP Address	Manual	Up
FastEthernet0/0	192.168.1.1	YES	up
FastEthernet0/1	192.168.1.2	YES	down
Vlan1	unassigned	YES	down

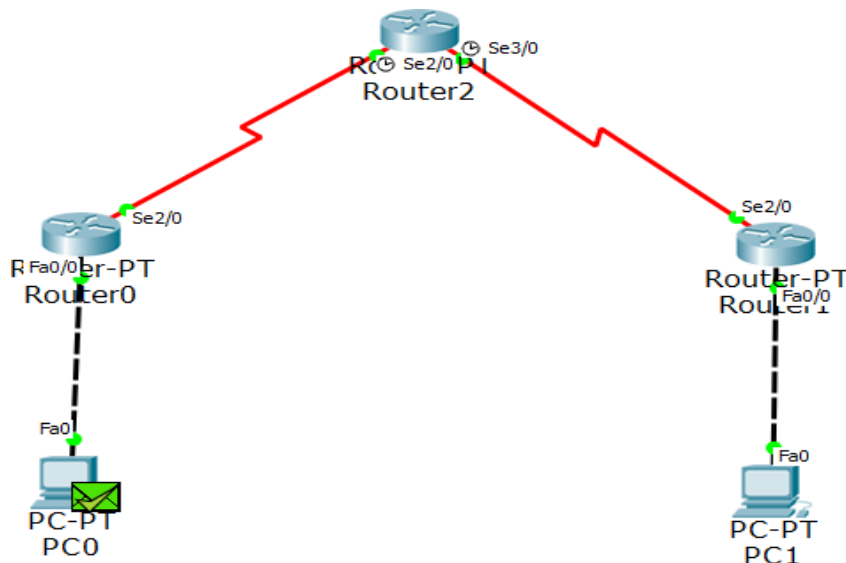
Screenshots/ Output:



```
Router2
Physical Config CLI
IOS Command Line Interface

Router(config-if)#clock rate 64000
Router(config-if)#no shut
Router(config-if)#exit
Router(config)#router rip
Router(config-router)#version 2
Router(config-router)#network 11.0.0.0
Router(config-router)#network 12.0.0.0
Router(config-router)#exit
Router(config)#interface Se3/0
Router(config-if)#encapsulation ppp
Router(config-if)#
%LINEPROTO-5-UPDOWN: Line protocol on Interface
Serial3/0, changed state to down

Router(config-if)#clock rate 64000
Router(config-if)#no shut
Router(config-if)#exit
Router(config)#exit
Router#
%SYS-5-CONFIG_I: Configured from console by console
```



Observation:

- RIP routing updates were successfully exchanged between routers, allowing each router to dynamically learn remote network routes through hop-count-based distance vector advertisements.
- The routing tables converged correctly, and successful ping tests confirmed end-to-end

connectivity maintained by periodic RIP updates and route propagation.

Program 7

Aim of the program:

To construct a VLAN and make the PCs communicate among a VLAN

Procedure and topology:

8/10/25.

PAGE NO:
 DATE:

7) Configure OSPF Routing Protocol.
(Open shortest Path First).

R1

R2

R3

PC0

PC1

R0

```
# conf t
# router ospf 1
# network 192.168.55.0 0.0.0.255 area 0
# network 176.16.0.0 0.0.0.255 area 0
# exit
```

R1

```
# conf t
# network 192.168.55.0 0.0.0.255 area 0
# network 192.168.44.0 0.0.0.255 area 0
# exit
```

R2

```
# conf t
# router ospf 1
# network 176.16.10.0 0.0.255.255 area 0
# network 10.10.22.0 0.0.0.255 area 0
# exit
```

PC-0: Ping 10.10.22.2
Success [sent 4 Received 4 cost 0]

Screenshots/ Output:

The image displays a network topology and two screenshots of network configuration and testing.

Network Topology: A central switch (SWITCH1) is connected to four PCs (PC-PT PC0, PC1, PC2, PC3) and a router (Router1). The switch has four ports (Fa0/1, Fa0/2, Fa0/3, Fa0/4) connected to the PCs. The router has a single port (Fa0/0/41) connected to the switch. The switch is also connected to the router via its Fa0/5 port.

Router1 CLI Screenshot: The screenshot shows the configuration of the router's Fa0/0.1 interface. The configuration includes enabling the interface, setting the IP address to 192.168.20.1, and enabling the line protocol. The output shows the interface is up and the line protocol is up.

```
Router1
Physical Config CLI
IOS Command Line Interface
APPL1 completed.
Exiting....
Router#enable
Router#config t
Enter configuration commands, one per line. End with
CNTL/Z.
Router(config)#interface Fa0/0.1
Router(config-subif)#
%LINK-5-CHANGED: Interface FastEthernet0/0.1, changed
state to up
%LINEPROTO-5-UPDOWN: Line protocol on Interface
FastEthernet0/0.1, changed state to up
Router(config-subif)#encapsulation dot1q 20
Router(config-subif)#ip address 192.168.20.1
% Incomplete command.
Router(config-subif)#ip address 192.168.20.1
255.255.0
Router(config-subif)#no shut
Router(config-subif)#exit
Router(config)#
```

PC0 Command Prompt Screenshot: The screenshot shows the results of ping tests from PC0. The first test is to 192.168.1.3, which is successful. The second test is to 192.168.20.2, which fails with a 100% loss.

```
PC0
Physical Config Desktop Custom Interface
Command Prompt
Packet Tracer PC Command Line 1.0
PC>ping 192.168.1.3
Pinging 192.168.1.3 with 32 bytes of data:
Reply from 192.168.1.3: bytes=32 time=1ms TTL=128
Reply from 192.168.1.3: bytes=32 time=1ms TTL=128
Reply from 192.168.1.3: bytes=32 time=0ms TTL=128
Reply from 192.168.1.3: bytes=32 time=0ms TTL=128
Ping statistics for 192.168.1.3:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 1ms, Average = 0ms
PC>ping 192.168.20.2
Pinging 192.168.20.2 with 32 bytes of data:
Request timed out.
Request timed out.
Request timed out.
Request timed out.
Ping statistics for 192.168.20.2:
    Packets: Sent = 4, Received = 0, Lost = 4 (100%
loss),
PC>
```

Observation:

- VLAN segmentation successfully separated broadcast domains, and switch ports were correctly assigned to their respective VLAN IDs using access mode configuration.
- Inter-VLAN communication was achieved through the Layer-3 device, and successful ping tests confirmed proper VLAN membership, tagging, and routing functionality.

Program 8

Aim of the program:

To construct a WLAN and make the nodes communicate wirelessly

Procedure and topology:

15/10/25.

8) To construct a VLAN & make the PCs communi-
-cating among a VLAN.

IP Addresses:

PC	IP Address
PC0	192.168.1.2
PC1	192.168.1.3
PC2	192.168.1.4
PC3	192.168.1.5
PC4	192.168.1.6
PC5	192.168.1.7

VLAN10	VLAN20	VLAN30
Fa0/1 PC0	Fa0/2 PC1	Fa0/3 PC2
Fa0/1 PC3	Fa0/2 PC4	Fa0/3 PC5

Switch1 > CLI.

```

Switch> enable
Switch# conf t
Switch (config) # int Fa0/1
Switch (config-if) # switchport access vlan 10
% Access VLAN does not exist. (creating vlan)
Switch (config-if) # int Fa0/2
  
```

Switch (config-if) # switchport access vlan 20
creating vlan 20

Switch (config-if) # int Fa0/3
Switch (config-if) # switchport access vlan 30
creating vlan 30

Switch (config-if) # int Fa0/4
Switch (config-if) # switchport mode trunk

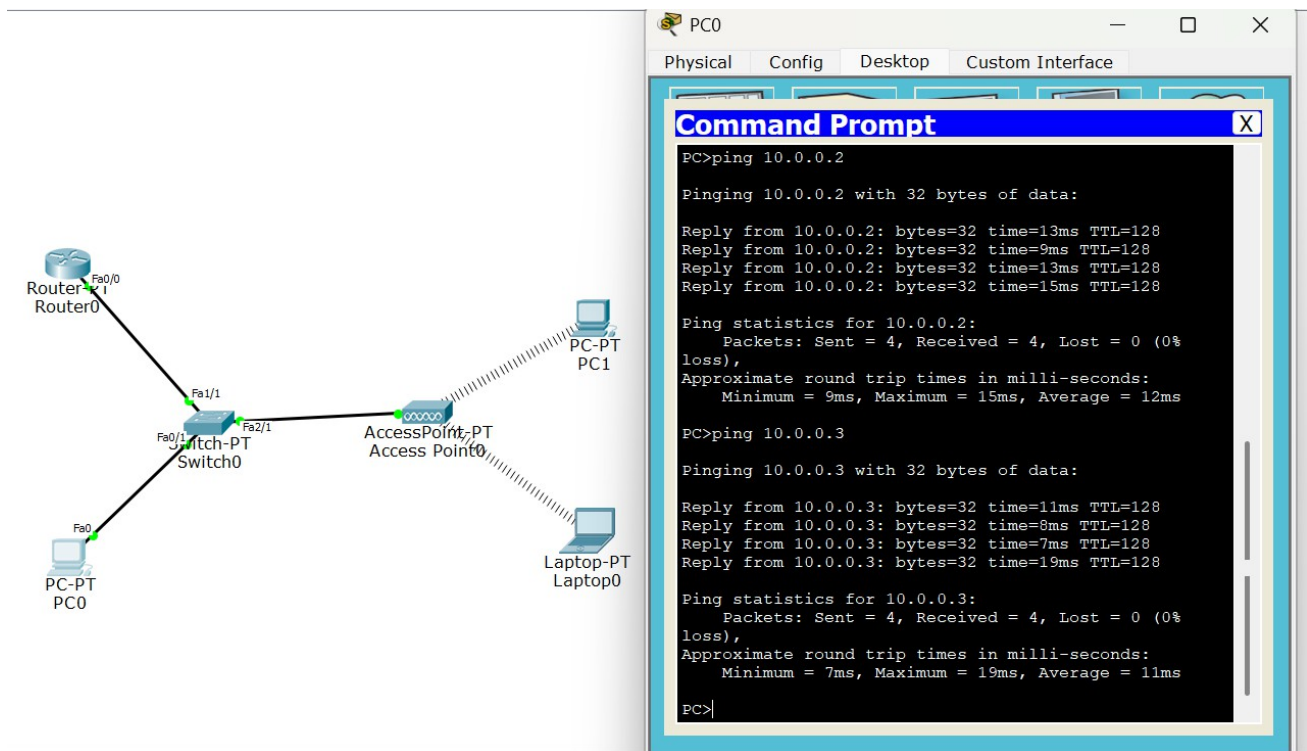
Switch2 > CLI

```

> enable
# conf t
# int Fa0/1
# switchport access vlan 10
# int Fa0/2
# switchport access vlan 20
# int Fa0/3
# switchport access vlan 30
# int Fa0/4
# switchport mode trunk trunk
# exit
  
```

- Before setting a VLAN
 - Ping message from PC0 → PC2 Successful
 - Ping message from PC0 → PC1 Successful
- After setting VLAN
 - Ping message from PC0 → PC2 Failed
 - Ping message from PC0 → PC1 Successful

Screenshots/ Output:



Observation:

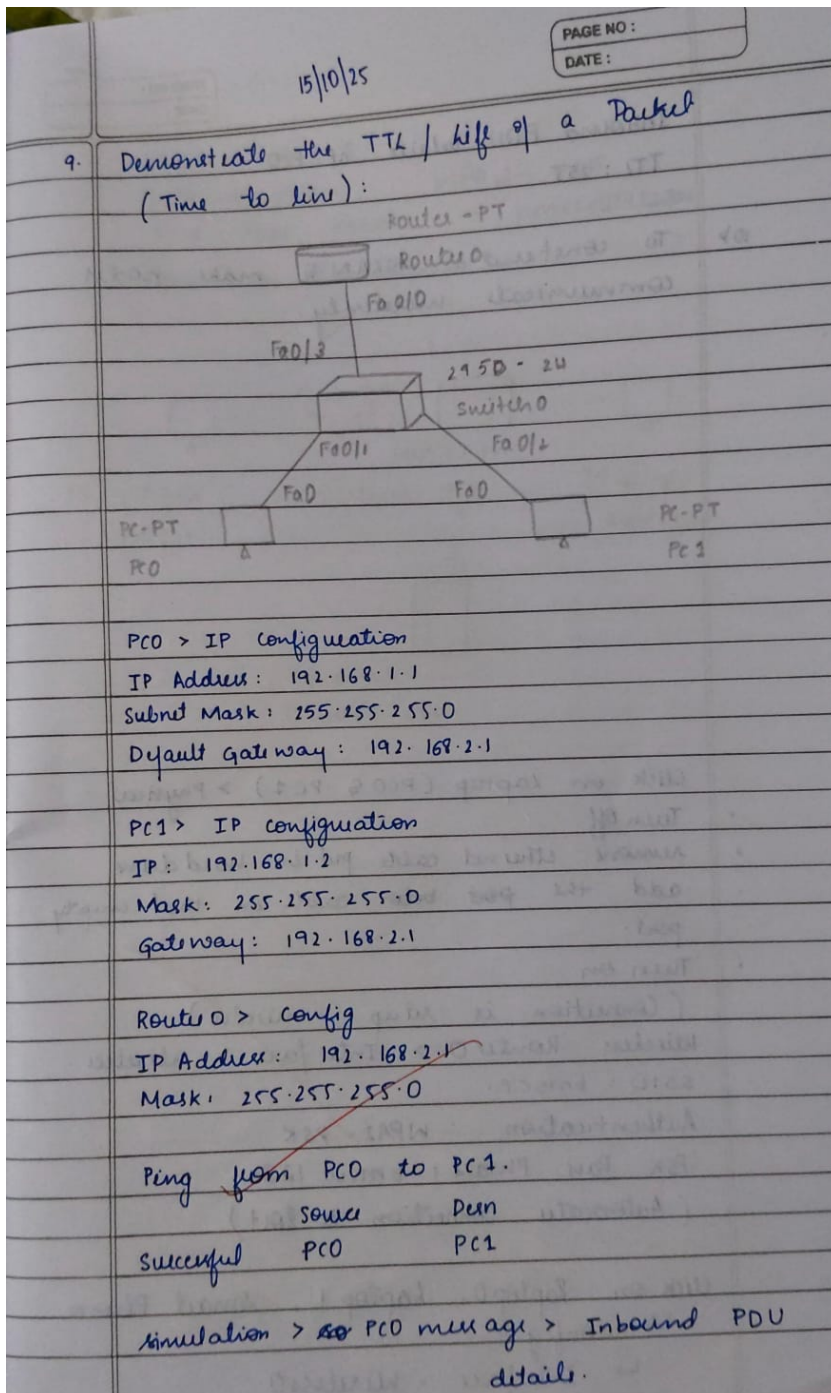
- The WLAN configuration enabled wireless nodes to associate with the access point using the configured SSID and security settings, confirming proper authentication and signal coverage.
- Successful ping communication between wireless devices verified stable wireless Connectivity.

Program 9

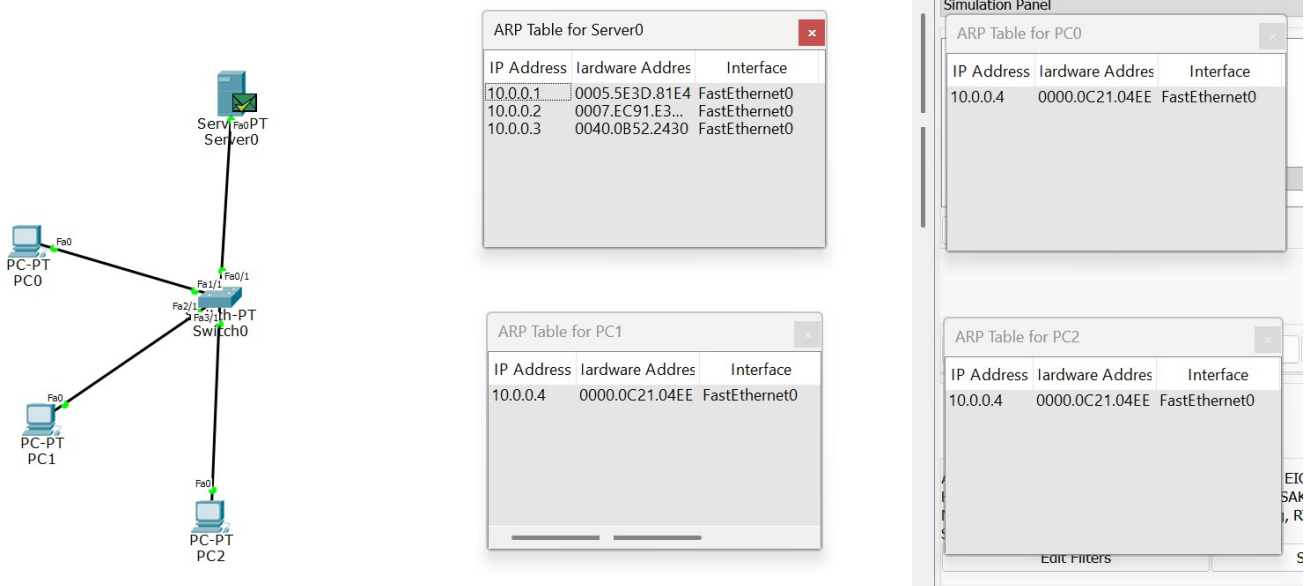
Aim of the program:

To construct simple LAN and understand the concept and operation of Address Resolution Protocol (ARP)

Procedure and topology:



Screenshots/ Output:



Observation:

- ARP successfully resolved the destination IP address to its corresponding MAC address, as seen from ARP request and reply exchanges between LAN hosts.
- The populated ARP tables and successful ping communication confirmed correct layer-2 addressing, frame forwarding, and basic LAN operation.

Program 10

Aim of the program:

Configure OSPF routing protocol

Procedure and topology:

PAGE NO:
 DATE:

Inbound PDU Details at PC0
ITL: 255

10x To configure a WLAN & make nodes communicate wirelessly.

WRT300N

Laptop0 PT
Laptop0

Wireless Router0

Laptop1 PT
Laptop1

SmartPhone0 PT
SmartPhone0

Click on Laptop (PC0 & PC2) > Physical

- Turn Off
- remove ethernet cable put in hard disk
- add the port bto cable in that empty port.
- Turn On

(Connection is setup - wireless).

Wireless Router0 > Interface > Wireless.

SSID: bmsce.

Authentication : WPA2-PSK

PSK Pass Phrase : bmsce1234

(Automatic connection is lost)

Click on Laptop0, Laptop1, SmartPhone

- ↳ Config
- ↳ Interface - Wireless0

PAGE NO:
 DATE:

SSID: bmsce

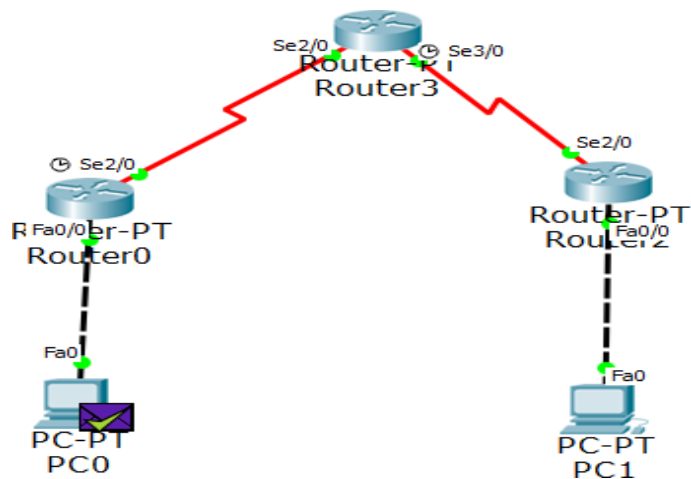
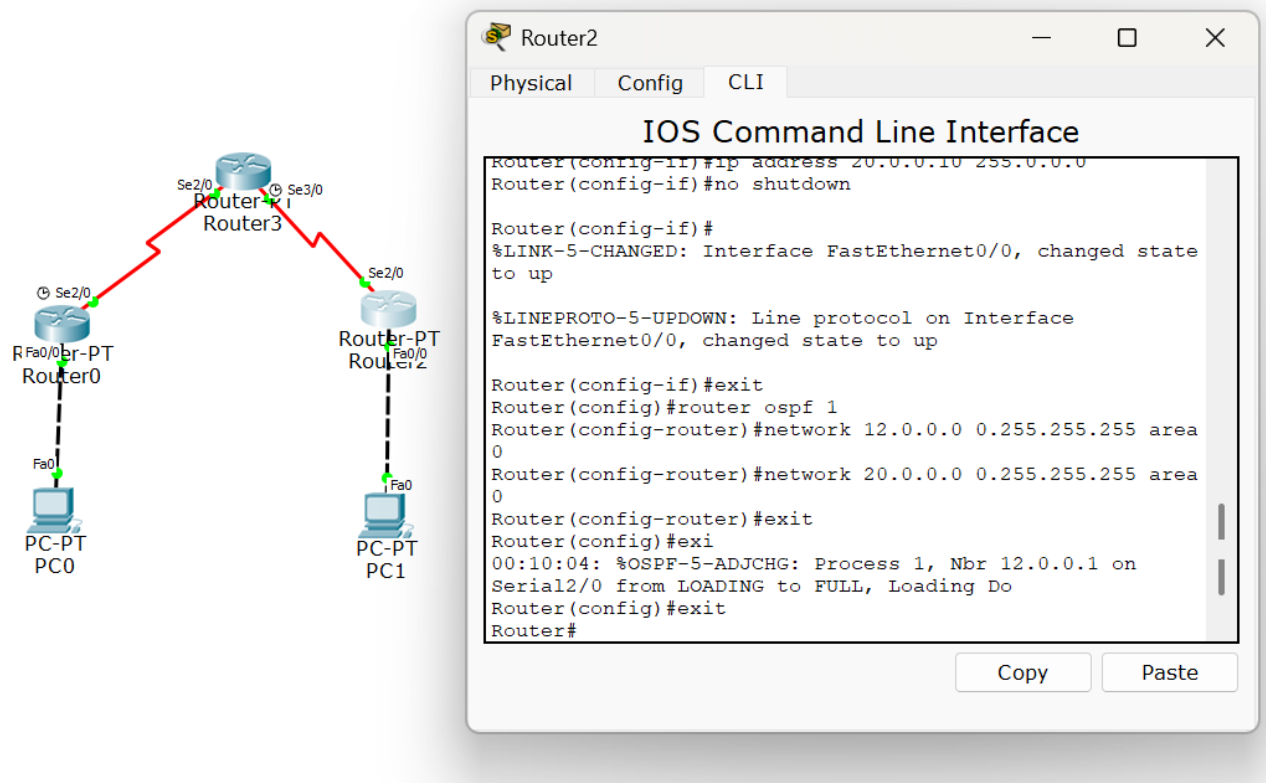
Authentication : WPA2-PSK

PSK Pass Phrase : bmsce1234

Connection Established.

15/10/2020

Screenshots/ Output:



Observation:

- OSPF successfully established neighbour adjacencies and exchanged LSAs, allowing routers to build a synchronized link-state database across the OSPF area.
- The routing tables converged using SPF calculations, and successful pings confirmed efficient

path selection and dynamic route learning through OSPF.

Program 11

Aim of the program:

Demonstrate the TTL/ Life of a Packet

Procedure and topology:

```
29/10/25.
PAGE NO:
DATE:

11. Leaky Bucket.

#include <stdio.h>
#include <string.h>

int min (int x, int y) {
    return (x < y) ? x : y;
}

int main() {
    int deep = 0, mini, maxi, cap, count = 0, i;
    int inp[25], process;
    printf("Enter the bucket size: \n");
    scanf("%d", &cap);
    printf("Enter the processing rate: \n");
    scanf("%d", &process);
    printf("Enter the number of seconds you want to simulate: \n");
    scanf("%d", &nsec);

    for (i = 0; i < nsec; i++) {
        printf("Enter the size of the packet entering at %d sec: \n", i+1);
        scanf("%d", &inp[i]);
    }

    printf("\n Second | Packet Received | Packet set | Packet left | Dropped \n");
    printf("-----\n");

    for (i = 0; i < nsec; i++) {
        if (count > cap) {
            deep = count - cap;
            count = cap;
        }
        printf("%6d | %15d |", i+1, inp[i]);
        mini = min(count, process);
        printf("%11d |", mini);
        count -= mini;
        printf("%11d | %7d \n", count, deep);
        deep = 0;
    }

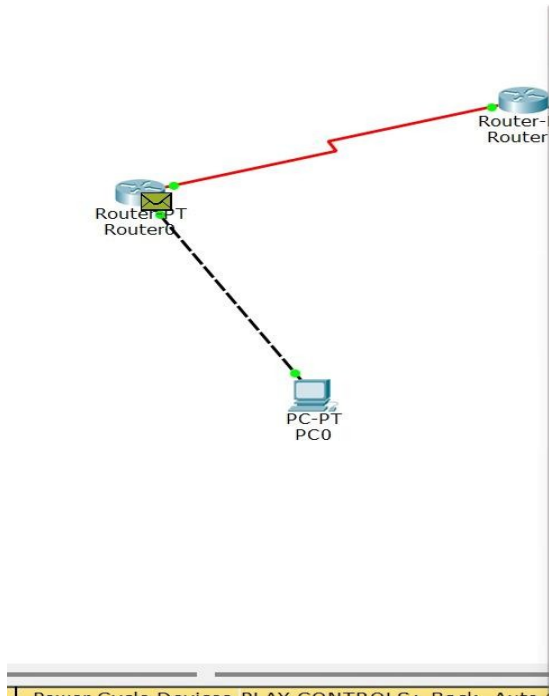
    while (count != 0) {
        if (count > cap) {
            deep = count - cap;
            count = cap;
        }
        printf("%6d | %15d |", i+1, 0);
        mini = min(count, process);
        printf("%11d |", mini);
        count -= mini;
        printf("%11d | %7d \n", count, deep);
        deep = 0;
    }

    return 0;
}

OUTPUT:
Enter the bucket size : 5
Enter the processing rate : 2
Enter the number of seconds you want to simulate : 3
```

Second	Packet Received	Packet set	Packet left	Dropped
1	5	2	3	0
2	4	2	3	2
3	3	2	3	1
4	0	2	1	0
5	0	1	0	0

Screenshots/ Output:



PDU Information at Device: Router0

OSI Model Inbound PDU Details Outbound PDU Details

PDU Formats

Ethernet II

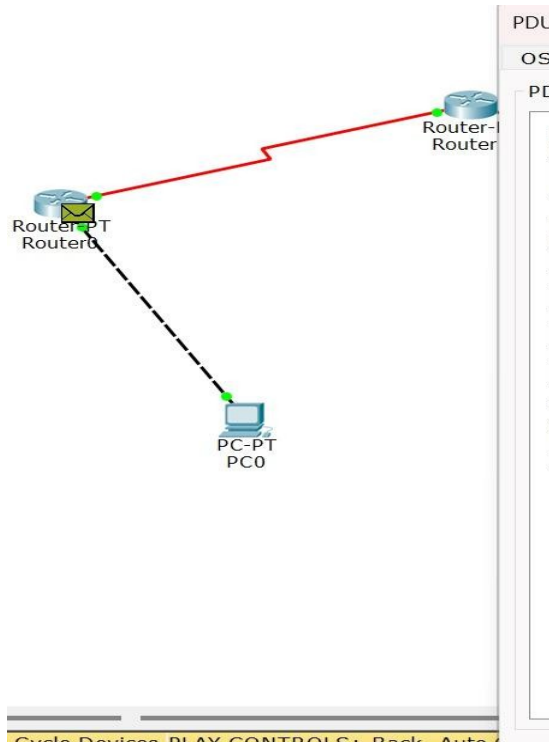
PREAMBLE:		DEST MAC:		SRC MAC:	
101010...101					
TYP	DATA (VARIABLE LENGTH)			FCS	
E:					

IP

ID: 0x3		DSCP:		TL: 28	
				0x0	
TTL: 255		PRO: 0x1		CHKSUM	
SRC IP: 10.0.0.10					
DST IP: 20.0.0.10					
OPT: 0x0				0x0	
DATA (VARIABLE LENGTH)					

ICMP

TYPE:		CODE:		CHECKSUM	
ID: 0x4				SEQ NUMBER: 3	



PDU Information at Device: Router0

OSI Model Inbound PDU Details Outbound PDU Details

PDU Formats

HDLC

FL	AD	CONTR	DATA:		FCS:	FL
G:	R:	OL:	(VARIABLE)		0x0	G:

IP

ID: 0x3		DSCP:		TL: 28	
				0x0	
TTL: 254		PRO: 0x1		CHKSUM	
SRC IP: 10.0.0.10					
DST IP: 20.0.0.10					
OPT: 0x0				0x0	
DATA (VARIABLE LENGTH)					

ICMP

TYPE:		CODE:		CHECKSUM	
ID: 0x4				SEQ NUMBER: 3	

Observation:

- The TTL field in the IP header decreased by one at each router hop, demonstrating its role in preventing packets from looping indefinitely in the network.

Program 12

Aim of the program:

Configure IP address to routers in packet tracer. Explore the following messages: ping responses, destination unreachable, request timed out, reply

Procedure and topology:

PAGE NO :
DATE :

Q2. ~~Write a~~ Error detection using CRC-CCITT (16 bit)

F: 1101011011 G: 10011

Add (G-1) zeroes. (XOR for subtraction)

110000101

10011 | 11010110110000

- 10011 ↓

010011

10011 ↓

0000010110

- 10011 ↓

0010100

10011 ↓

001110 - Remainder.

There is error in the transmission

F: 100100 G: $x^3 + x^2 + 1 \Rightarrow 1101$

(G-1): 3 zeroes.

1101 | 100100000 | 111101

1101 ↓

01000

1101 ↓

01010

1101 ↓

01110

1101 ↓

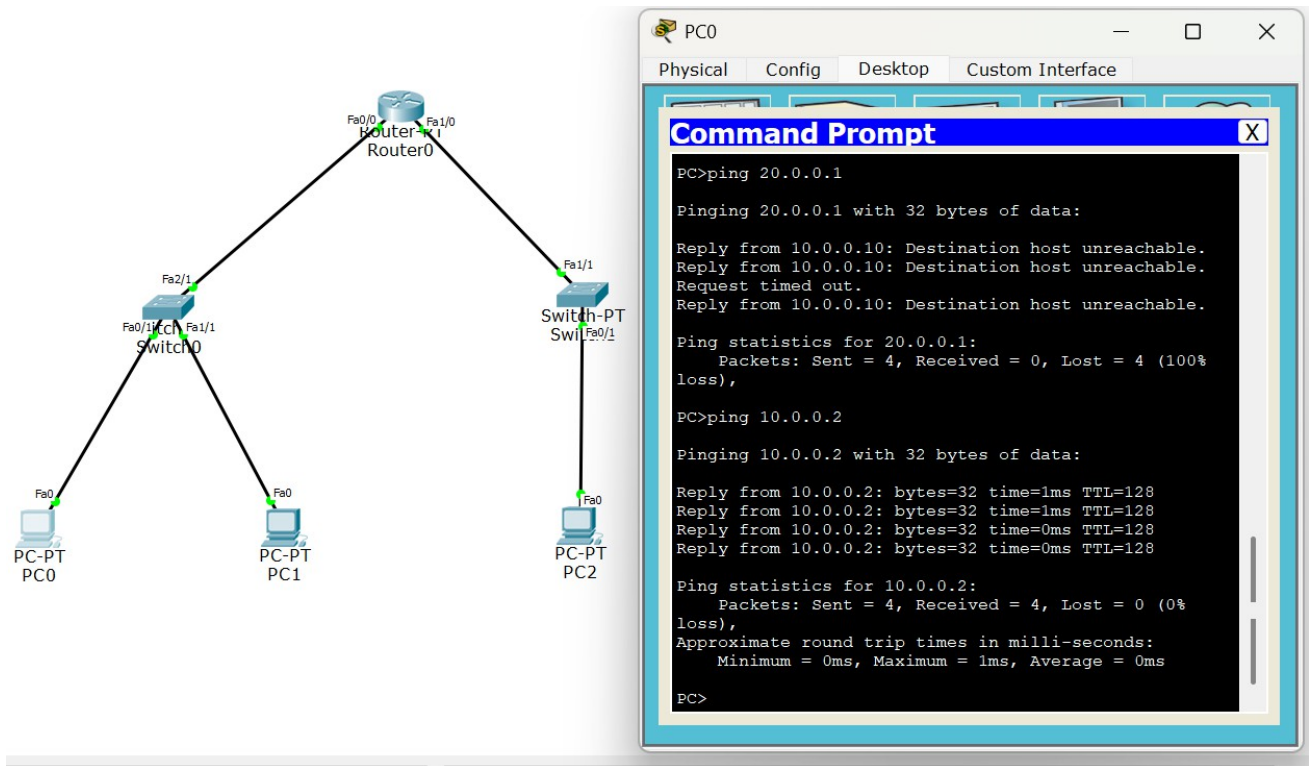
001100

1101 ↓

0001

Q2. 29/10/25

Screenshots/ Output:



Observation:

- Proper IP addressing on router interfaces enabled successful ICMP echo and echo-reply communication, confirming correct Layer-3 configuration and reachability.
- “Destination Unreachable” and “Request Timed Out” responses occurred when routes or interfaces were misconfigured, demonstrating how routers handle missing paths and non-responsive hosts.

CYCLE 2:

Program 13

Aim of the program:

Write a program for congestion control using Leaky bucket algorithm

Procedure and topology:

PAGE NO. : DATE :	PAGE NO. : DATE :
13. Write a program in C for client server communication using TCP or IP sockets to make client send the name of the file & server to send back the contents of the requested file if present. ALGORITHM (client side) 1) sfd = create a socket with the socket(...) system call. 2) connect the socket to the address of the server using connect (sfd, ...) system call. The IP address of the server machine & port no. of the server service need to be provided. 3) Read file name for standard input by n = read (stdio, buffer, sizeof (buffer)) 4) write file name to the socket using write (sfd, buffer, n). 5) read file contents from the socket by m = read (sfd, buffer1, sizeof (buffer1)) 6) display file contents to standard output by write (stdio, buffer1, m) 7) go to step 5 if m > 0 8) close socket by close (sfd) ALGORITHM (server side) 1) sfd = create a socket with socket (...) sys call. 2) bind the socket to an address using the bind (sfd, ...) system call. If not sure of machine IP address, keep the structure member s_addr to INADDR_ANY. Assign a port no. b/w 3000 & 5000 to in_port	3) listen for connections with the listen (sfd, ...) sys call 4) sfd = accept a connection with the accept (sfd, ...) sys call. 5) read the file name by socket 6) open the file by fd = open (buffer) 7) read the contents of file 8) write the file content to socket 9) go to step 7 if m > 0 10) close (sfd) 12/11/25

Screenshots/ Output:

```
Output
Enter bucket size: 4
Enter outgoing size: 1
Enter number of inputs: 7
Enter the incoming packet size: 3
Bucket buffer size 3 out of 4
After outgoing 2 packets left out of 4 in buffer
Enter the incoming packet size: 2
Bucket buffer size 4 out of 4
After outgoing 3 packets left out of 4 in buffer
Enter the incoming packet size: 1
Bucket buffer size 4 out of 4
After outgoing 3 packets left out of 4 in buffer
Enter the incoming packet size: 4
Dropped 3 packets
Bucket buffer size 4 out of 4
After outgoing 3 packets left out of 4 in buffer
Enter the incoming packet size: 0
Bucket buffer size 3 out of 4
After outgoing 2 packets left out of 4 in buffer
Enter the incoming packet size: 0
Bucket buffer size 2 out of 4
After outgoing 1 packets left out of 4 in buffer
Enter the incoming packet size: 0
Bucket buffer size 1 out of 4
After outgoing 0 packets left out of 4 in buffer
```

Observation:

- The leaky bucket mechanism regulated the outgoing packet flow at a constant rate, preventing sudden traffic bursts from overwhelming the network.
- Packets exceeding the bucket capacity were dropped, demonstrating effective congestion control by smoothing traffic and enforcing rate-limiting.

Program 14

Aim of the program:

Write a program for error detecting code using CRC-CCITT (16-bits).

Procedure and topology:

PAGE NO: _____
DATE: _____

4. Address Resolution Protocol (ARP).

ARP is used to map an IP address to a MAC Address. ARP is used to get data link layer address, MAC address with the help of IP Address.

```
graph TD
    Server[server  
IP: 192.168.11.4] --- Switch[2950-24 switch]
    Switch --- PC0[PC0  
IP: 192.168.11.1]
    Switch --- PC1[PC1  
IP: 192.168.11.2]
    Switch --- PC2[PC2  
IP: 198.168.11.3]
```

DNS Server: 192.168.11.4 192.168.11.2 192.168.11.4 192.168.11.4

- send a simple PDU from PC0 to PC1. select the (inspect icon) then click on PC0 & the ARP table. The details of the packet will be visible.
- PC0 > command prompt > arp -a then > ping 192.168.11.4.
- simulation: After sending a simple PDU from PC0 to PC1. click on PC0. Outbound details, the MAC Address will be 0. The capture/forward then click on the packet, Inbound details, the MAC Address (Target) will have a value.

Screenshots/ Output:

```
Output

Enter message bits: 101100
Enter polynomial g(x): 1001

Padded data (Message + zeros): 101100000
CRC bits (remainder): 001
Transmitted message: 101100001

Enter received bits: 101100001

No Error detected. Message OK.

=== Code Execution Successful ===
```

Observation:

- The CRC-CCITT (16-bit) algorithm correctly generated a checksum for the transmitted data, ensuring reliable detection of single-bit and burst errors.
- Intentional error tests produced mismatched CRC values at the receiver, confirming accurate error detection through polynomial division.

Program 15

Aim of the program:

Using TCP/IP sockets, write a client-server program to make client sending the file name and the server to send back the contents of the requested file if present.

Procedure and topology:

15. Configure RIP routing Protocol to transfer packet from Node A to Node B.

IP Address : 192.168.1.2 IP : 192.168.2.2.
Default Gateway : 192.168.1.1 Default Gateway : 192.168.2.1

R1 : Config : FastEthernet0/0 → Turn ON
IP Address : 192.168.1.1

R2 : Config : FastEthernet0/0 → Turn ON
IP Address : 192.168.2.1

R1 : Config : Serial2/0 → Turn ON
Clock Rate : 64000
IP Address : 10.10.0.2

R2 : Config : Serial2/0 → Turn ON
Clock Rate : 64000.
IP Address : 10.10.0.3

R1 : Config → RIP → Network 192.168.0.0 → Add.
Network 10.0.0.0 → Add.

Settings → Save (NVRAM).

R2 : Config → RIP → Network 192.168.2.0 → Add
Network 10.0.0.0 → Add

CMD Router (config) # exit
Router (config) # exit
Router #

Building configuration [OK]
and msg packet from PC0 to PC1
successful.

Screenshots/ Output:

```
● PS D:\CN stuff\TCP> python client.py
Enter filename to request: test.txt

--- File Content ---

Hello from server side
○ PS D:\CN stuff\TCP> 
```

```
PS D:\CN stuff\TCP> python --version
● Python 3.12.6
● PS D:\CN stuff\TCP> python server.py
Server is listening on port 8080 ...
Connected by: ('127.0.0.1', 65258)
○ PS D:\CN stuff\TCP> 
```

Observation:

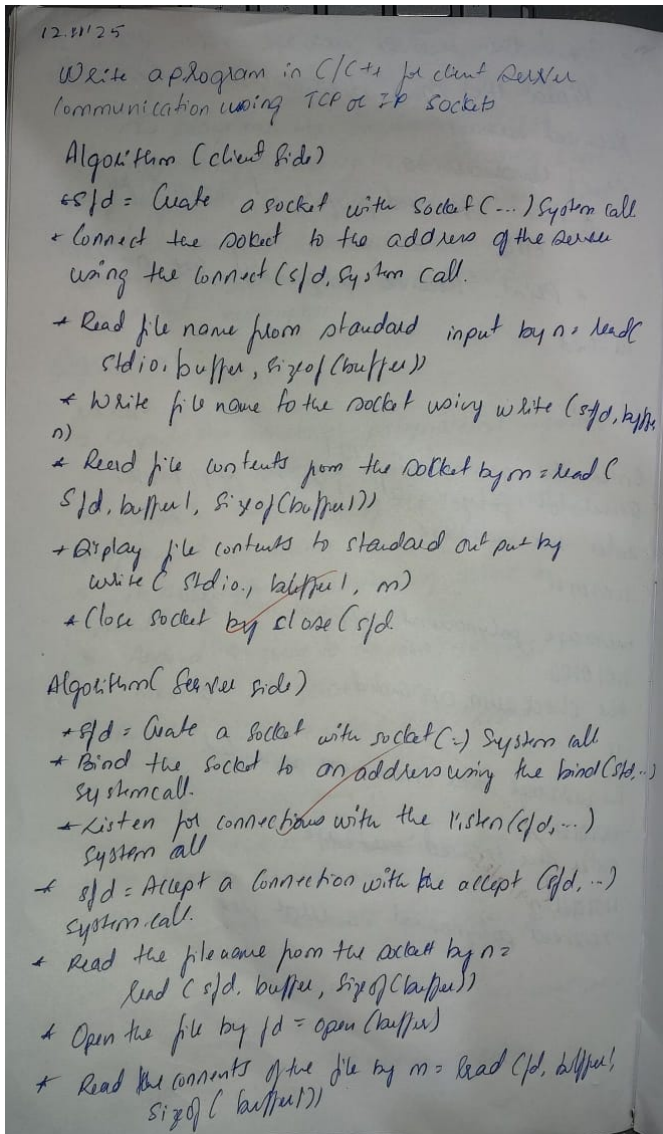
- The TCP client successfully established a reliable connection with the server and transmitted the requested filename using stream-oriented communication.
- The server correctly retrieved and returned the file contents over the same TCP session, demonstrating reliable data transfer, acknowledgment handling, and error-free delivery.

Program 16

Aim of the program:

Using UDP sockets, write a client-server program to make client sending the file name and the server to send back the contents of the requested file if present.

Procedure and topology:



Screenshots/ Output:

```
● PS D:\CN stuff\UDP> python server.py
UDP Server is ready ...
Requested file: test.txt
○ PS D:\CN stuff\UDP> █
```

```
● PS D:\CN stuff\UDP> python client.py
Enter filename to request: test.txt

--- File Content ---

Welcome to UDP file server
○ PS D:\CN stuff\UDP> █
```

Observation:

- The UDP client successfully sent the filename as a connectionless datagram, demonstrating non-reliable, low-overhead message transfer without session establishment.
- The server responded with the file contents using UDP packets, and correct reception validated functional data exchange despite the absence of acknowledgment and retransmission mechanisms.